

UNIT-III

HASHING

Last-Minute Revision Handout

Hashing

One-Way Function: Converts any input into a fixed-length hash.

Key Properties

- **Avalanche Effect:** Hard to find input from hash
- **Pre-Image Resistance:** Hard finding input from hash
- **Collision Resistance:** Extremely hard to find two inputs with same hash
- **Second Pre-Image Resistance:** Hard finding different input with same hash

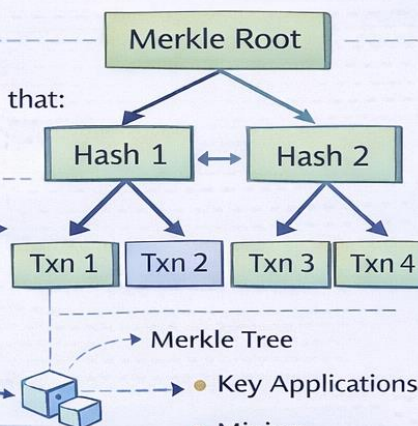
Mathematics

Given y , hard to find x such that:

$$H(x) = y$$

Blockchain

- Mining
- Merkle Trees
- Block Linking



- Key Applications
- Mining
- Merkle Trees
- Block Linking

DIGITAL SIGNATURES

(ECDSA)

Digital Signatures: ECDSA

- A Digital Signature = **Cryptographic** mechanism verifying authenticity and integrity

Signature: Private Key →  Signature (r, s) 

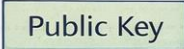
Mathematics

Given y , hard to find x such that:

$$H(x) = y$$



Signature Verification

- Calculate $w = s^{-1} \bmod n$
- Calculate $u_1 = H(m) \cdot w \bmod n$ → 
- Calculate $u_2 = r \cdot w \bmod n$
- Calculate $X = u_1 G + u_2 Q$
- Signature is valid if $X_x \bmod n = r$ ✓

Key Uses

-  Authorize Transactions
-  Prevent Double Spending
-  Secure Smart Contracts



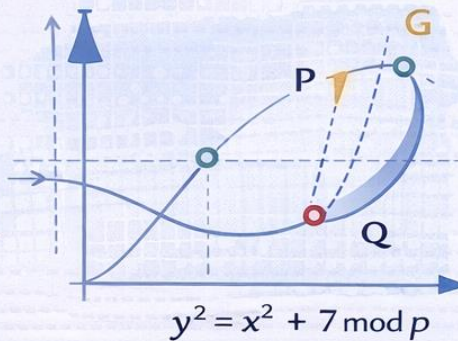
ELLIPTIC CURVE CRYPTOGRAPHY: ECC

Last-Minute Revision Handout

Elliptic Curve Cryptography: ECC

- Public-Key Cryptography Using Elliptic Curve Algebra
Curve: secp256k1 used by Bitcoin

$$y^2 = x^3 + 7 \bmod p$$

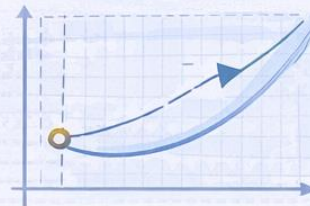


Key Components

- Private Key: d
- Public Key: $Q = dG$
- Point Addition: $P + Q \rightarrow R$

Key Components

- Private Key: d
- Public Key: $Q = dG$
- Point Addition: $P + Q \rightarrow R$
- Scalar Multiplication: $Q = dG$



Applications in Blockchain

- ✓ Bitcoin & Ethereum Wallets
- ✓ Digital Signatures
- ✓ Deterministic Wallets (BIP32)

SYMMETRIC & ASYMMETRIC ENCRYPTION

Last-Minute Revision Handout

Symmetric Encryption

- Single key (**secret key**) shared between sender & receiver



Features

- Same key for encryption & decryption
- Very fast, efficient
- Good for large data

Asymmetric Encryption

- Public key for encryption & private key for decryption

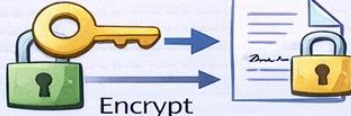


Features

- Different keys for encryption & decryption
- More secure**, works for digital signatures
- Slower, heavier computation

In Cryptocurrency

-  Private Keys in Wallets
-  Digital Signatures (Asymmetric)
-  Encrypted Connections



Common Algorithms

Symmetric	Asymmetric
AES, DES	RSA, ECC
Very Fast	Slower
128-256 bits	256-521 bits

VRFs

(Verifiable Random Functions)

What is a VRF?

- A **cryptographic** function that produces a **verifiable random** output based on an input and a private key, similar to a digital signature.



Anyone with the public key can **verify** that the output is indeed random and correctly derived from the input.

Mathematics

Options: Elliptic Curve, RSA

Pseudo-random output:

$$Y = \text{VRFH}_s(x)$$

Proof: $\pi = \text{hash}(k, H_s(x))$



Output Verified

Key Properties



Verifiable: Anyone can check the output is correct



Random: Output Y is unique & unpredictable



Collision Resistant: Simple hashes cannot mimic the Y



Unpredictable: Hard to find output before input is known

Applications in Blockchain



Randomness for Proof of Stake (PoS)



Selecting Block Producers & Validators



On-Chain Gaming & Lotteries

Symmetric

AES, DES

RSA, ECC



Chainlink



Algorand



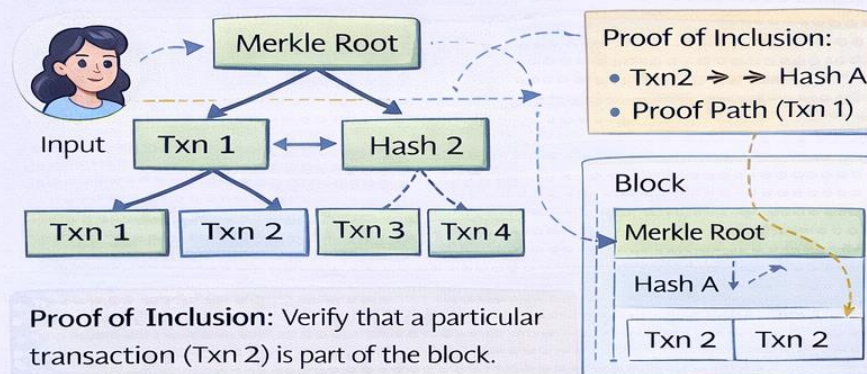
Polkadot

MERKLE TREES

Last-Minute Revision Handout

What is a Merkle Tree?

- A **Merkle Tree** is a cryptographic tree structure used to efficiently and securely verify the contents of a large set of data.



Mathematics

- Every leaf node is a hash of the data
 $\text{Hash A} = H(\text{Txn 2})$
- Each non-leaf node is the hash of the concatenation of its child hashes

Key Properties

- Efficient:** Anyone can check transaction is part of the tree
- Tamper-Proof:** Any change in data changes the Merkle Root
- Scalable:** Efficiently verify large data sets

Applications in Blockchain

- Mining
- Simplifies Block Validation
- Efficient Light Clients

Full Node	Light Client
Data	Header + Merkle Proof
Txn 2	Txn 2

Tip: Saves Light Clients from downloading full blocks, by verifying with Merkle Proofs.

HYBRID ENCRYPTION

Last-Minute Revision Handout

What is Hybrid Encryption?

- Combines Symmetric & Asymmetric Encryption to leverage the strengths of both.



Symmetric

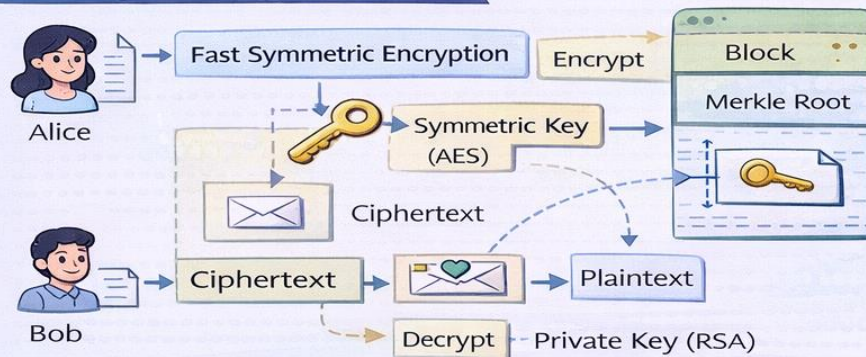
- Fast, Efficient



Asymmetric

- Secure Key Exchange

How Hybrid Encryption Works



Key Advantages



Efficiency: Fast data encryption with secure key exchange



Security: Transmissions protected by strong **public-key** cryptography



Scalability: Suited for large-scale systems & many recipients

Did You Know?



Secure Messaging



E-commerce Transactions (HTTPS)



Secure File Sharing



Secure Data Storage



Random Symmetric Key is used only once (unique per session), ensuring **Forward Secrecy**: past sessions remain safe even if a long-term private key is compromised.

Common Hybrid Encryption Algorithms

- AES + RSA
- AES + ECC